

Section 78 Editorial — Master Suffix Structures Mixed Contest

Each solution starts from the uncompressed object, identifies the bottleneck, and states the invariant that justifies the compressed structure.

A. Minimum Rotation

How to find it

Compare two candidate starts and skip every start inside the losing matched block; none can beat the winner. This is Booth's elimination argument.

Correctness

After a mismatch at offset k , the losing candidate and the next k starts have the same obstructing prefix relation and cannot be minimal. Thus discarded starts are safe, and one survivor is globally minimal.

Complexity

$O(n)$ time and $O(1)$ auxiliary memory.

At a mismatch, advance the losing start by the matched length plus one; if both candidate indices coincide, advance one again. Compare through the doubled string modulo n without constructing all rotations. Test unary strings, periodic strings with several equal minimum rotations, and a winner near the last index.

Reference C++

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    int n = s.size(), i = 0, j = 1, k = 0;
    while (i < n && j < n && k < n) {
        char a = s[(i + k) % n], b = s[(j + k) % n];
        if (a == b) {
            ++k;
            continue;
        }
        if (a > b) {
            i = i + k + 1;
            if (i == j)
                ++i;
        } else {
            j = j + k + 1;
            if (i == j)
                ++j;
        }
        k = 0;
    }
```

```

}
int at = min(i, j);
cout << s.substr(at) + s.substr(0, at) << '\n' << at + 1 << '\n';
}

```

Reference Python

```

import sys

def main():
    s = sys.stdin.buffer.readline().decode().strip()
    n = len(s)
    first, second, offset = 0, 1, 0
    while first < n and second < n and offset < n:
        a, b = s[(first + offset) % n], s[(second + offset) % n]
        if a == b:
            offset += 1
            continue
        if a > b:
            first += offset + 1
            if first == second:
                first += 1
        else:
            second += offset + 1
            if first == second:
                second += 1
        offset = 0
    start = min(first, second)
    print(s[start:] + s[:start])
    print(start + 1)

if __name__ == "__main__":
    main()

```

B. Repetition Spectrum

How to find it

Treat adjacent suffix-array positions as edges weighted by LCP. Add edges from high to low; a DSU component of size k certifies a prefix occurring in those k suffixes.

Correctness

At threshold L , components are exactly maximal groups connected by LCP values at least L , hence exactly groups sharing a length- L prefix. Recording merge sizes and suffix-maximizing over k gives every at-least threshold.

Complexity

$O(n \log n)$ after suffix-array construction and $O(n)$ memory.

Process all edges of the same LCP weight as one threshold before recording answers, or equivalent merge ordering must be justified. A final suffix maximum over occurrence counts is necessary because a substring occurring s times also satisfies every smaller threshold. Compare with literal substring-frequency maps on tiny periodic strings.

Reference C++

```
#include <bits/stdc++.h>
using namespace std;
vector<int> suffix_array(const string &s) {
    int n = s.size();
    vector<int> sa(n), r(n), nr(n);
    iota(sa.begin(), sa.end(), 0);
    for (int i = 0; i < n; ++i)
        r[i] = (unsigned char)s[i];
    for (int k = 1;; k *= 2) {
        sort(sa.begin(), sa.end(), [&](int a, int b) {
            return pair{r[a], a + k < n ? r[a + k] : -1} <
                pair{r[b], b + k < n ? r[b + k] : -1};
        });
        nr[sa[0]] = 0;
        for (int i = 1; i < n; ++i) {
            int a = sa[i - 1], b = sa[i];
            nr[b] = nr[a] + (pair{r[a], a + k < n ? r[a + k] : -1} <
                pair{r[b], b + k < n ? r[b + k] : -1});
        }
        r = nr;
        if (r[sa.back()] == n - 1)
            break;
    }
    return sa;
}

vector<int> lcp_array(const string &s, const vector<int> &sa) {
    int n = s.size(), h = 0;
    vector<int> rank(n), lcp(max(0, n - 1));
    for (int i = 0; i < n; ++i)
        rank[sa[i]] = i;
    for (int i = 0; i < n; ++i) {
        int q = rank[i];
        if (!q)
            continue;
        int j = sa[q - 1];
        while (i + h < n && j + h < n && s[i + h] == s[j + h])
            ++h;
        lcp[q - 1] = h;
        if (h)
            --h;
    }
    return lcp;
}

struct DSU {
    vector<int> p, z;
    DSU(int n) : p(n), z(n, 1) { iota(p.begin(), p.end(), 0); }
    int f(int x) { return p[x] == x ? x : p[x] = f(p[x]); }
    int unite(int a, int b) {
        a = f(a);
        b = f(b);
        if (a == b)
            return z[a];
        if (z[a] < z[b])
            swap(a, b);
        p[b] = a;
        return z[a] += z[b];
    }
};

int main() {
    ios::sync_with_stdio(false);
```

```

cin.tie(nullptr);
string s;
cin >> s;
int n = s.size();
auto sa = suffix_array(s), lcp = lcp_array(s, sa);
vector<array<int, 3>> e;
for (int i = 0; i < n - 1; ++i)
    e.push_back({lcp[i], i, i + 1});
sort(e.rbegin(), e.rend());
DSU d(n);
vector<int> ans(n + 1);
ans[1] = n;
for (auto [w, a, b] : e) {
    int size = d.unite(a, b);
    ans[size] = max(ans[size], w);
}
for (int k = n - 1; k; --k)
    ans[k] = max(ans[k], ans[k + 1]);
for (int k = 1; k <= n; ++k)
    cout << ans[k] << " \n"[k == n];
}

```

Reference Python

```

def suffix_array(s):
    n = len(s)
    sa = list(range(n))
    rank = list(map(ord, s))
    step = 1
    while True:
        sa.sort(
            key=lambda i: (rank[i], rank[i + step] if i + step < n else -1)
        )
        new_rank = [0] * n
        for j in range(1, n):
            a, b = sa[j - 1], sa[j]
            left = (rank[a], rank[a + step] if a + step < n else -1)
            right = (rank[b], rank[b + step] if b + step < n else -1)
            new_rank[b] = new_rank[a] + (left < right)
        rank = new_rank
        if rank[sa[-1]] == n - 1:
            return sa
        step *= 2

def lcp_array(s, sa):
    n = len(s)
    rank = [0] * n
    for i, start in enumerate(sa):
        rank[start] = i
    lcp = [0] * (n - 1)
    height = 0
    for i in range(n):
        place = rank[i]
        if place == 0:
            continue
        j = sa[place - 1]
        while (
            i + height < n
            and j + height < n
            and s[i + height] == s[j + height]

```

```

    ):
        height += 1
        lcp[place - 1] = height
        height = max(0, height - 1)
    return lcp

def main():
    import sys

    s = sys.stdin.buffer.readline().decode().strip()
    n = len(s)
    sa = suffix_array(s)
    edges = sorted(
        ((w, i, i + 1) for i, w in enumerate(lcp_array(s, sa))), reverse=True
    )
    parent = list(range(n))
    size = [1] * n

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    answer = [0] * (n + 1)
    answer[1] = n
    for weight, a, b in edges:
        a, b = find(a), find(b)
        if a != b:
            if size[a] < size[b]:
                a, b = b, a
            parent[b] = a
            size[a] += size[b]
            answer[size[a]] = max(answer[size[a]], weight)
    for k in range(n - 1, 0, -1):
        answer[k] = max(answer[k], answer[k + 1])
    print(*answer[1:])

if __name__ == "__main__":
    main()

```

C. K-th Substring With Multiplicity

How to find it

A SAM path still identifies a distinct substring, but its terminal state's occurrence count is the multiplicity of that path label. Weight each path endpoint by this count.

Correctness

Transition blocks partition the sorted multiset by next letter. Within a block, the immediate label contributes `occ[target]` copies before all longer extensions, exactly matching lexicographic multiset order.

Complexity

$O(n \cdot \text{alphabet})$ time and memory, with saturated 64-bit counts.

Occurrence counts flow from longer SAM states to suffix links before weighted path DP. During reconstruction, the immediate label contributes `occ[next]` identical copies, not one. Saturate every addition above the largest requested rank. Test overlapping occurrences and a unary string, where multiplicities vary at every path depth.

Reference C++

```
#include <bits/stdc++.h>
using namespace std;
struct SAM {
    struct Node {
        array<int, 26> next;
        int link = -1, len = 0;
        long long occ = 0;
        Node() { next.fill(-1); }
    };
    vector<Node> st;
    int last = 0;
    SAM() { st.emplace_back(); }
    void add(int c) {
        int cur = st.size();
        st.emplace_back();
        st[cur].len = st[last].len + 1;
        st[cur].occ = 1;
        int p = last;
        while (p != -1 && st[p].next[c] == -1)
            st[p].next[c] = cur, p = st[p].link;
        if (p == -1)
            st[cur].link = 0;
        else {
            int q = st[p].next[c];
            if (st[p].len + 1 == st[q].len)
                st[cur].link = q;
            else {
                int clone = st.size();
                st.push_back(st[q]);
                st[clone].len = st[p].len + 1;
                st[clone].occ = 0;
                while (p != -1 && st[p].next[c] == q)
                    st[p].next[c] = clone, p = st[p].link;
                st[q].link = st[cur].link = clone;
            }
        }
        last = cur;
    }
    SAM(const string &s) : SAM() {
        for (char c : s)
            add(c - 'a');
    }
    vector<int> order() const {
        vector<int> o(st.size());
        iota(o.begin(), o.end(), 0);
        sort(o.begin(), o.end(),
            [&](int a, int b) { return st[a].len < st[b].len; });
        return o;
    }
    void occurrences() {
        auto o = order();
        for (int i = (int)o.size() - 1; i; --i)
            st[st[o[i]].link].occ += st[o[i]].occ;
    }
};
```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    long long k;
    cin >> s >> k;
    SAM a(s);
    a.occurrences();
    const long long CAP = 4000000000000000LL;
    auto o = a.order();
    vector<long long> dp(a.st.size());
    for (int z = (int)o.size() - 1; z >= 0; --z) {
        int v = o[z];
        for (int u : a.st[v].next)
            if (u != -1) {
                long long add = min(CAP, a.st[u].occ + dp[u]);
                dp[v] = dp[v] >= CAP - add ? CAP : dp[v] + add;
            }
    }
    if (k > dp[0]) {
        cout << "IMPOSSIBLE\n";
        return 0;
    }
    string ans;
    int v = 0;
    while (true) {
        for (int c = 0; c < 26; ++c) {
            int u = a.st[v].next[c];
            if (u == -1)
                continue;
            long long block = min(CAP, a.st[u].occ + dp[u]);
            if (k > block) {
                k -= block;
                continue;
            }
            ans += char('a' + c);
            if (k <= a.st[u].occ) {
                cout << ans << '\n';
                return 0;
            }
            k -= a.st[u].occ;
            v = u;
            break;
        }
    }
}

```

Reference Python

```

class SAM:
    def __init__(self, s):
        self.next = [{}]*26
        self.link = [-1]*26
        self.length = [0]*26
        self.occ = [0]*26
        last = 0
        for char in s:
            current = len(self.next)
            self.next.append({})
            self.link.append(0)
            self.length.append(self.length[last] + 1)

```

```

        self.occ.append(1)
        parent = last
        while parent != -1 and char not in self.next[parent]:
            self.next[parent][char] = current
            parent = self.link[parent]
        if parent == -1:
            self.link[current] = 0
        else:
            target = self.next[parent][char]
            if self.length[parent] + 1 == self.length[target]:
                self.link[current] = target
            else:
                clone = len(self.next)
                self.next.append(self.next[target].copy())
                self.link.append(self.link[target])
                self.length.append(self.length[parent] + 1)
                self.occ.append(0)
                while (
                    parent != -1 and self.next[parent].get(char) == target
                ):
                    self.next[parent][char] = clone
                    parent = self.link[parent]
                self.link[target] = self.link[current] = clone
        last = current

def order(self):
    return sorted(range(len(self.next)), key=self.length.__getitem__)

def occurrences(self):
    order = self.order()
    for state in reversed(order[1:]):
        self.occ[self.link[state]] += self.occ[state]

def main():
    import sys

    raw_s, raw_k = sys.stdin.buffer.read().split()
    automaton = SAM(raw_s.decode())
    automaton.occurrences()
    k = int(raw_k)
    paths = [0] * len(automaton.next)
    for state in reversed(automaton.order()):
        paths[state] = sum(
            automaton.occ[target] + paths[target]
            for target in automaton.next[state].values()
        )
    if k > paths[0]:
        print("IMPOSSIBLE")
        return
    answer = []
    state = 0
    while True:
        for char, target in sorted(automaton.next[state].items()):
            block = automaton.occ[target] + paths[target]
            if k > block:
                k -= block
                continue
            answer.append(char)
            if k <= automaton.occ[target]:
                print("".join(answer))
                return
            k -= automaton.occ[target]

```



```

        state = target
        break

if __name__ == "__main__":
    main()

```

D. Common Distinct Substrings

How to find it

While scanning the second string, record the best accepted match ending in each SAM state, then propagate capped values down suffix links.

Correctness

A state represents all lengths from $\text{len}(\text{link})+1$ through $\text{len}(\text{state})$. After propagation, precisely the lengths up to $\text{best}[\text{state}]$ occur in both strings, giving the stated interval contribution without duplication.

Complexity

$O(|a|+|b|)$ time apart from fixed transition initialization.

Streaming visits alone are insufficient: a shorter suffix state may never be the final state of the scan despite representing common strings. Propagate best lengths in decreasing len , clamping by the suffix-link state's maximum. For state v , contribute $\max(0, \text{best}[v] - \text{len}[\text{link}[v]])$. Test disjoint strings, complete containment, and repeated borders.

Reference C++

```

#include <bits/stdc++.h>
using namespace std;
struct SAM {
    struct Node {
        array<int, 26> next;
        int link = -1, len = 0;
        long long occ = 0;
        Node() { next.fill(-1); }
    };
    vector<Node> st;
    int last = 0;
    SAM() { st.emplace_back(); }
    void add(int c) {
        int cur = st.size();
        st.emplace_back();
        st[cur].len = st[last].len + 1;
        st[cur].occ = 1;
        int p = last;
        while (p != -1 && st[p].next[c] == -1)
            st[p].next[c] = cur, p = st[p].link;
        if (p == -1)
            st[cur].link = 0;
        else {
            int q = st[p].next[c];
            if (st[p].len + 1 == st[q].len)
                st[cur].link = q;
            else {
                int clone = st.size();
                st.push_back(st[q]);
                st[clone].len = st[p].len + 1;

```

```

        st[clone].occ = 0;
        while (p != -1 && st[p].next[c] == q)
            st[p].next[c] = clone, p = st[p].link;
        st[q].link = st[cur].link = clone;
    }
}
last = cur;
}
SAM(const string &s) : SAM() {
    for (char c : s)
        add(c - 'a');
}
vector<int> order() const {
    vector<int> o(st.size());
    iota(o.begin(), o.end(), 0);
    sort(o.begin(), o.end(),
        [&](int a, int b) { return st[a].len < st[b].len; });
    return o;
}
void occurrences() {
    auto o = order();
    for (int i = (int)o.size() - 1; i; --i)
        st[st[o[i]].link].occ += st[o[i]].occ;
}
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string a, b;
    cin >> a >> b;
    SAM s(a);
    vector<int> best(s.st.size());
    int v = 0, len = 0;
    for (char ch : b) {
        int c = ch - 'a';
        while (v && s.st[v].next[c] == -1)
            v = s.st[v].link, len = min(len, s.st[v].len);
        if (s.st[v].next[c] != -1)
            v = s.st[v].next[c], ++len;
        else
            v = 0, len = 0;
        best[v] = max(best[v], len);
    }
    auto o = s.order();
    for (int i = (int)o.size() - 1; i; --i) {
        int x = o[i], p = s.st[x].link;
        best[p] = max(best[p], min(best[x], s.st[p].len));
    }
    long long ans = 0;
    for (int x = 1; x < (int)s.st.size(); ++x)
        ans += max(0, min(s.st[x].len, best[x]) - s.st[s.st[x].link].len);
    cout << ans << '\n';
}

```

Reference Python

```

class SAM:
    def __init__(self, s):
        self.next = [{}]
```

```

self.length = [0]
self.occ = [0]
last = 0
for char in s:
    current = len(self.next)
    self.next.append({})
    self.link.append(0)
    self.length.append(self.length[last] + 1)
    self.occ.append(1)
    parent = last
    while parent != -1 and char not in self.next[parent]:
        self.next[parent][char] = current
        parent = self.link[parent]
    if parent == -1:
        self.link[current] = 0
    else:
        target = self.next[parent][char]
        if self.length[parent] + 1 == self.length[target]:
            self.link[current] = target
        else:
            clone = len(self.next)
            self.next.append(self.next[target].copy())
            self.link.append(self.link[target])
            self.length.append(self.length[parent] + 1)
            self.occ.append(0)
            while (
                parent != -1 and self.next[parent].get(char) == target
            ):
                self.next[parent][char] = clone
                parent = self.link[parent]
            self.link[target] = self.link[current] = clone
    last = current

def order(self):
    return sorted(range(len(self.next)), key=self.length.__getitem__)

def occurrences(self):
    order = self.order()
    for state in reversed(order[1:]):
        self.occ[self.link[state]] += self.occ[state]

def main():
    import sys

    a, b = (part.decode() for part in sys.stdin.buffer.read().split())
    automaton = SAM(a)
    best = [0] * len(automaton.next)
    state = length = 0
    for char in b:
        while state and char not in automaton.next[state]:
            state = automaton.link[state]
            length = min(length, automaton.length[state])
        if char in automaton.next[state]:
            state = automaton.next[state][char]
            length += 1
        else:
            state = length = 0
        best[state] = max(best[state], length)
    for state in reversed(automaton.order()[1:]):
        parent = automaton.link[state]
        best[parent] = max(
            best[parent], min(best[state], automaton.length[parent])

```

```

    )
    answer = 0
    for state in range(1, len(automaton.next)):
        low = automaton.length[automaton.link[state]]
        answer += max(0, min(automaton.length[state], best[state]) - low)
    print(answer)

if __name__ == "__main__":
    main()

```

E. Palindrome Frequency Value

How to find it

Create one node per palindrome, count visits to the current longest suffix, then propagate counts from longer nodes to their palindromic suffix links.

Correctness

Every occurrence is first credited to the longest palindromic suffix at its endpoint. Suffix-link propagation credits that endpoint to every shorter palindromic suffix exactly once.

Complexity

$O(n \cdot \text{alphabet})$ memory and amortized $O(n)$ traversal.

Initialize direct occurrence count only when the node is the current longest palindromic suffix at an endpoint, then propagate nodes in decreasing palindrome length. Multiply each distinct palindrome's length by its final occurrence count only after propagation. Do not count the two roots. Palindromic borders and unary strings are the strongest small tests.

Reference C++

```

#include <bits/stdc++.h>
using namespace std;
struct Eertree {
    struct N {
        array<int, 26> to;
        int len = 0, link = 0;
        long long occ = 0;
        N() { to.fill(0); }
    };
    vector<N> t;
    string s;
    int last = 1;
    Eertree() {
        t.resize(2);
        t[0].len = -1;
        t[0].link = 0;
        t[1].len = 0;
        t[1].link = 0;
    }
    bool add(char ch) {
        int c = ch - 'a', pos = s.size();
        s += ch;
        int p = last;
        while (pos - 1 - t[p].len < 0 || s[pos - 1 - t[p].len] != ch)
            p = t[p].link;
        if (t[p].to[c]) {

```

```

        last = t[p].to[c];
        ++t[last].occ;
        return false;
    }
    int v = t.size();
    t.emplace_back();
    t[v].len = t[p].len + 2;
    t[p].to[c] = v;
    if (t[v].len == 1)
        t[v].link = 1;
    else {
        int q = t[p].link;
        while (pos - 1 - t[q].len < 0 || s[pos - 1 - t[q].len] != ch)
            q = t[q].link;
        t[v].link = t[q].to[c];
    }
    last = v;
    t[v].occ = 1;
    return true;
}
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    Eertree e;
    for (char c : s)
        e.add(c);
    vector<int> o(e.t.size());
    iota(o.begin(), o.end(), 0);
    sort(o.begin(), o.end(),
        [&](int a, int b) { return e.t[a].len < e.t[b].len; });
    long long ans = 0;
    for (int i = (int)o.size() - 1; i >= 2; --i) {
        int v = o[i];
        ans = max(ans, e.t[v].occ * e.t[v].len);
        e.t[e.t[v].link].occ += e.t[v].occ;
    }
    cout << ans << '\n';
}

```

Reference Python

```

class Eertree:
    def __init__(self):
        self.next = [{}, {}]
        self.length = [-1, 0]
        self.link = [0, 0]
        self.occ = [0, 0]
        self.text = []
        self.last = 1

    def add(self, char):
        position = len(self.text)
        self.text.append(char)
        node = self.last
        while (
            position - 1 - self.length[node] < 0
            or self.text[position - 1 - self.length[node]] != char

```

```

):
    node = self.link[node]
    if char in self.next[node]:
        self.last = self.next[node][char]
        self.occ[self.last] += 1
        return False
    created = len(self.next)
    self.next.append({})
    self.length.append(self.length[node] + 2)
    self.link.append(1)
    self.occ.append(1)
    self.next[node][char] = created
    if self.length[created] > 1:
        suffix = self.link[node]
        while (
            position - 1 - self.length[suffix] < 0
            or self.text[position - 1 - self.length[suffix]] != char
        ):
            suffix = self.link[suffix]
        self.link[created] = self.next[suffix][char]
    self.last = created
    return True

def main():
    import sys

    tree = Eertree()
    for char in sys.stdin.buffer.readline().decode().strip():
        tree.add(char)
    order = sorted(
        range(2, len(tree.next)), key=tree.length.__getitem__, reverse=True
    )
    answer = 0
    for node in order:
        answer = max(answer, tree.length[node] * tree.occ[node])
        tree.occ[tree.link[node]] += tree.occ[node]
    print(answer)

if __name__ == "__main__":
    main()

```

F. Multi-Archive Commonality

How to find it

Use the first archive's SAM as the candidate universe. For every other archive, scan matches and suffix-link-propagate the maximum supported length per state. The k -th largest archive support is that state's feasible length.

Correctness

Every common candidate occurs in the first archive and belongs to one state. Per-archive propagation computes its maximum valid length; requiring at least k archives is exactly taking the k -th largest support.

Complexity

$O(\text{total length} + m \cdot n + m \cdot n \log m)$ with $m \leq 20$, and $O(mn)$ memory.

Clear per-archive streaming evidence without erasing accumulated evidence from previous archives. After link propagation, collect each archive's supported length for a state and choose the required order statistic, capped below by `len[link]+1`. Test duplicate archives, an archive sharing only short suffixes, and qualification thresholds one and `m`.

Stress-testing and carry-forward

The random oracles enumerate rotations, substring occurrence maps, and palindromic intervals directly. The key distinction to preserve in future problems is whether a compressed state represents one string, an interval of string lengths, or an occurrence class; most wrong solutions mix two of those meanings.

Reference C++

```
#include <bits/stdc++.h>
using namespace std;
struct SAM {
    struct Node {
        array<int, 26> next;
        int link = -1, len = 0;
        long long occ = 0;
        Node() { next.fill(-1); }
    };
    vector<Node> st;
    int last = 0;
    SAM() { st.emplace_back(); }
    void add(int c) {
        int cur = st.size();
        st.emplace_back();
        st[cur].len = st[last].len + 1;
        st[cur].occ = 1;
        int p = last;
        while (p != -1 && st[p].next[c] == -1)
            st[p].next[c] = cur, p = st[p].link;
        if (p == -1)
            st[cur].link = 0;
        else {
            int q = st[p].next[c];
            if (st[p].len + 1 == st[q].len)
                st[cur].link = q;
            else {
                int clone = st.size();
                st.push_back(st[q]);
                st[clone].len = st[p].len + 1;
                st[clone].occ = 0;
                while (p != -1 && st[p].next[c] == q)
                    st[p].next[c] = clone, p = st[p].link;
                st[q].link = st[cur].link = clone;
            }
        }
        last = cur;
    }
    SAM(const string &s) : SAM() {
        for (char c : s)
            add(c - 'a');
    }
    vector<int> order() const {
        vector<int> o(st.size());
        iota(o.begin(), o.end(), 0);
        sort(o.begin(), o.end(),
            [&](int a, int b) { return st[a].len < st[b].len; });
        return o;
    }
};
```

```

void occurrences() {
    auto o = order();
    for (int i = (int)o.size() - 1; i; --i)
        st[st[o[i]].link].occ += st[o[i]].occ;
}

};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int m, k;
    cin >> m >> k;
    vector<string> a(m);
    for (auto &x : a)
        cin >> x;
    SAM s(a[0]);
    int z = s.st.size();
    vector<vector<int>> seen(z, vector<int>(m));
    for (int v = 0; v < z; ++v)
        seen[v][0] = s.st[v].len;
    auto order = s.order();
    for (int id = 1; id < m; ++id) {
        vector<int> best(z);
        int v = 0, len = 0;
        for (char ch : a[id]) {
            int c = ch - 'a';
            while (v && s.st[v].next[c] == -1)
                v = s.st[v].link, len = min(len, s.st[v].len);
            if (s.st[v].next[c] != -1)
                v = s.st[v].next[c], ++len;
            else
                v = 0, len = 0;
            best[v] = max(best[v], len);
        }
        for (int j = z - 1; j; j--) {
            int x = order[j], p = s.st[x].link;
            best[p] = max(best[p], min(best[x], s.st[p].len));
        }
        for (int x = 0; x < z; ++x)
            seen[x][id] = min(best[x], s.st[x].len);
    }
    int ans = 0;
    for (int x = 1; x < z; ++x) {
        nth_element(seen[x].begin(), seen[x].begin() + m - k, seen[x].end());
        ans = max(ans, seen[x][m - k]);
    }
    cout << ans << '\n';
}

```

Reference Python

```

class SAM:
    def __init__(self, s):
        self.next = [{}]
        self.link = [-1]
        self.length = [0]
        self.occ = [0]
        last = 0
        for char in s:
            current = len(self.next)
            self.next.append({})

```



```

self.link.append(0)
self.length.append(self.length[last] + 1)
self.occ.append(1)
parent = last
while parent != -1 and char not in self.next[parent]:
    self.next[parent][char] = current
    parent = self.link[parent]
if parent == -1:
    self.link[current] = 0
else:
    target = self.next[parent][char]
    if self.length[parent] + 1 == self.length[target]:
        self.link[current] = target
    else:
        clone = len(self.next)
        self.next.append(self.next[target].copy())
        self.link.append(self.link[target])
        self.length.append(self.length[parent] + 1)
        self.occ.append(0)
        while (
            parent != -1 and self.next[parent].get(char) == target
        ):
            self.next[parent][char] = clone
            parent = self.link[parent]
        self.link[target] = self.link[current] = clone
last = current

def order(self):
    return sorted(range(len(self.next)), key=self.length.__getitem__)

def occurrences(self):
    order = self.order()
    for state in reversed(order[1:]):
        self.occ[self.link[state]] += self.occ[state]

def main():
    import sys

    data = sys.stdin.buffer.read().split()
    m, k = map(int, data[:2])
    archives = [part.decode() for part in data[2:]]
    automaton = SAM(archives[0])
    states = len(automaton.next)
    seen = [[0] * m for _ in range(states)]
    for state in range(states):
        seen[state][0] = automaton.length[state]
    order = automaton.order()
    for archive_id, archive in enumerate(archives[1:], 1):
        best = [0] * states
        state = length = 0
        for char in archive:
            while state and char not in automaton.next[state]:
                state = automaton.link[state]
                length = min(length, automaton.length[state])
            if char in automaton.next[state]:
                state = automaton.next[state][char]
                length += 1
            else:
                state = length = 0
        best[state] = max(best[state], length)
    for state in reversed(order[1:]):
        parent = automaton.link[state]

```

```

        best[parent] = max(
            best[parent], min(best[state], automaton.length[parent])
        )
    for state in range(states):
        seen[state][archive_id] = min(
            best[state], automaton.length[state]
        )
    print(max(sorted(values)[-k] for values in seen[1:]))

if __name__ == "__main__":
    main()

```